

Homework Assignment #2
Due: September 18, 2024, by 11:59 pm

- **You must submit your assignment through the Crowdmark system.** You will receive by email an invitation through which you can submit your work in the form of separate PDF documents with your answers to each question of the assignment. To work with a partner, you and your partner must form a group on Crowdmark. Crowdmark does not enforce a limit on the size of groups. **The course policy that limits the size of each group to at most two remains in effect:** submissions by groups of more than two persons will not be graded.
- It is your responsibility to ensure that the PDF files you submit are legible. To this end, I encourage you to learn and use the LaTeX typesetting system, which is designed to produce high-quality documents that contain mathematical notation. You are not required to produce the PDF files you submit using LaTeX; you may produce it any way you wish, as long as the resulting document is legible.
- By virtue of submitting this assignment you (and your partner, if you have one) acknowledge that you are aware of the policy on homework collaboration for this course.^a
- For any question, you may use data structures and algorithms previously described in class, or in prerequisites of this course, without describing them. You may also use any result that we covered in class, or is in the assigned sections of the official course textbooks, by referring to it.
- Unless we explicitly state otherwise, you may describe algorithms in high-level pseudocode or point-form English, whichever leads to a clearer and simpler description. Do not provide executable code.
- Unless we explicitly state otherwise, you should justify your answers. Your paper will be graded based on **the correctness and efficiency of your answers, and the clarity, precision, and conciseness of your presentation.**

^a“In each homework assignment you may collaborate with at most one other student who is currently taking CSCC73. If you collaborate with another student on an assignment, you and your partner must submit only one copy of your solution, with both of your names. The solution will be graded in the usual way and both partners will receive the same mark. Collaboration involving more than two students is not allowed. **For help with your homework you may consult only the instructor, TAs, your homework partner (if you have one), your textbook, and your class notes. You may not consult any other source.**”

Question 1. (20 marks) A video production facility has n videos to process. The processing of each video i , $1 \leq i \leq n$, is done in two phases. In the first phase, the facility's editor edits the video. After the first phase of video i is finished, the edited video is processed by computer. The first phase of video i takes $f_i \geq 0$ units of time and the second phase takes $s_i \geq 0$ units of time.

The facility has a single video editor to do the first phase but it has n computers to do the second phase. So, the first phase must be done sequentially by the single editor, while the second phase can be done in parallel by the computers: As soon as the editor is finished editing a video, she feeds it into one of the free computers for the second phase, even as other computers may still be processing the videos whose first phase she completed earlier. The completion time of a video is the time by which its second phase is finished. The manager of the facility wants to find a schedule that **minimizes the maximum completion time over all n videos** — that is, the earliest time by which both phases of all videos have been completed. (This quantity is called the **makespan** of the schedule.)

Describe an efficient greedy algorithm that takes as input the set of videos and the length of their first and second phases $V = \{(i, f_i, s_i) : 1 \leq i \leq n\}$ and outputs an optimal schedule. In this case, a schedule is simply a permutation of the integers from 1 to n , indicating the order in which the editor processes the

videos. This determines the time by which each video finishes, and therefore the maximum completion time over all videos, which is the quantity we want to minimize.

Prove that your algorithm is correct and analyze its running time.

Question 2. (20 marks) A catering service receives n orders, each requiring *exactly one time unit* to complete. There is only one cook, so these orders must be completed one at a time; the cook starts work at time 0.

Order i , $1 \leq i \leq n$, has a **deadline** d_i (in time units) by which it must be completed, and a **profit** $p_i > 0$ that the catering service receives if the order is completed by the deadline; if the order is completed after the deadline there is no profit made and therefore it is not done at all. Note that several orders may have the same deadline (e.g., around lunch time). The catering service must decide on a schedule in which to do a subset of the orders, each completed by its deadline.

Describe an $O(n^2)$ greedy algorithm that takes as input the set of orders with their deadlines and profits $J = \{(i, d_i, p_i) : 1 \leq i \leq n\}$ and outputs a schedule of maximum profit. A schedule is a function S that maps each order i to a time slot j , where j is a positive integer, meaning that order i will be done during the time interval $(j-1, j)$; or to the time slot $+\infty$, meaning that order i will not be done. The schedule S cannot map two different orders to the same time slot (because there is only one cook who completes the orders, one at a time); and if it maps order i to time slot $j \neq 0$, then $j \leq d_i$ (since every order that is done must be finished by its deadline). The profit of schedule S is the sum of the orders that are done, i.e., $\sum_{1 \leq i \leq n \text{ \& } S(i) \neq \infty} p_i$.

Prove that your algorithm is correct and analyze its running time. (It can be done in $O(n \log n)$ time, but $\Theta(n^2)$ is good enough for the purposes of this question.)

Hint. No job can be scheduled after the maximum deadline. What job should be scheduled in the slot ending at that deadline?

Question 3. (15 marks) We are given a directed graph $G = (V, E)$, nodes $s, t \in V$, and an edge weight function **wt** such that $\mathbf{wt}(e) \geq 0$ for every $e \in E$. In addition, we are given a set E' of **potential edges**, i.e., pairs of nodes that are not edges in E ; and for each $e' \in E'$ we are given a non-negative weight $\mathbf{wt}(e')$.

We want to choose a **single** potential edge in E' that, if added to the graph, reduces as much as possible the weight of a shortest $s \rightarrow t$ path. (If there is no potential edge whose addition to G reduces the weight of the shortest $s \rightarrow t$ path, we can choose any potential edge.)

Describe an algorithm that solves this problem in $O(k + (m+n) \log n)$ time, where $n = |V|$, $m = |E|$, and $k = |E'|$. Assume that the graph is given in adjacency list form, and the set of potential edges (and their associated weights) is given as a list. Your algorithm **must be** by reduction of the given problem to the shortest-path problem, and using Dijkstra's algorithm to solve that.¹ Justify the correctness of your algorithm and analyze its running time.

¹Problem A is reduced to problem B if there is an algorithm that solves A using **as a black box** a solution to B . By “using as a black box” we mean that the reduction algorithm that solves A can make calls to a subroutine about which we know **only** that it solves B — not **how** it does so. (In particular, we cannot “look inside” the black box that solves B and change the way it works.) Thus, if A reduces to B and we know of an actual algorithm b that solves B , we can obtain an algorithm a that solves A by plugging b into the black box used by the reduction. Reductions are a very powerful algorithm design tool, as they allow us to solve new problems by leveraging algorithms that have already been devised to solve other problems. Certain problems play a particularly important role in computer science precisely because many other problems reduce to them. Shortest paths is one such problem. Later in the course we will encounter two other such problems: maximum flow and linear programming. When you take CSCC63 you will learn about a different use of reductions, namely as a tool for proving that certain problems are impossible to solve algorithmically, or are unlikely to be solved efficiently.

When you analyze the running time of a reduction algorithm you must take into account the cost of the calls to the black box. So, for example, if you know that B can be solved by an algorithm whose running time is $O(f(n))$, you must charge $O(f(\ell))$ for each use of the black box on an input of size ℓ by the reduction algorithm.

Additional question. (0 marks) This is a question to help solidify your understanding of Huffman's algorithm. You should work on it after the relevant lecture. **Do not submit your answer to this question.**

a. Give an example of a (small) set of symbols and their associated frequencies so that there is exactly one symbol with frequency $1/3$, all other symbols have frequency (strictly) less than $1/3$, and Huffman's algorithm may produce a codeword of length 1.

b. Prove that for any set of symbols, if every symbol has frequency (strictly) less than $1/3$, Huffman's algorithm cannot produce a codeword of length 1.